Universidad de Carabobo

Facultad de Ciencias y Tecnología (FACYT)

Departamento de Computación

Asignatura: Arquitectura del Computador

4to Semestre

SIMULADOR DE CACHÉ DE CORRESPONDENCIA DIRECTA Y ASOCIATIVA POR VÍAS EN SISTEMAS DE ARCHIVOS

Profesor:

José Canache

Estudiantes:

Fabian Ochoa

C.I.: 32.794.284

José Torres

C.I.: 31.530.657

Introducción

En el panorama actual de la arquitectura de computadores, la marcada disparidad de velocidad entre las unidades de procesamiento, la memoria principal y los dispositivos de almacenamiento secundario continúa siendo uno de los desafíos más críticos para el rendimiento global de los sistemas informáticos. Para mitigar esta latencia y evitar cuellos de botella operativos, los sistemas modernos implementan una jerarquía de memoria basada en memorias caché, cuyo diseño se fundamenta en los principios de localidad temporal y espacial de las referencias.

Por otro lado, los sistemas de archivos estructuran, organizan y gestionan el almacenamiento, recuperación y manipulación de los datos y metadatos en los soportes de almacenamiento secundario. La interacción entre los patrones de acceso generados por las operaciones de un sistema de archivos y la estructura de una memoria caché determina en gran medida la eficiencia del sistema. Sin embargo, evaluar empíricamente cómo diferentes configuraciones de caché —como el mapeo directo o la asociatividad por vías— responden ante diversas cargas de trabajo de archivos requiere de herramientas analíticas especializadas.

Este informe presenta el diseño, la implementación algorítmica y el análisis de rendimiento de un Simulador de Caché de Correspondencia Directa y Asociativa por Vías, desarrollado en C++ utilizando la Biblioteca Estándar de Plantillas (STL). El propósito principal de este simulador es procesar secuencias de direcciones de memoria, incorporar los perfiles estructurales de dos sistemas de archivos comparativos y cuantificar métricas esenciales de desempeño —tales como aciertos (hits), fallos (misses) y la tasa de aciertos (hit ratio)—. Mediante este enfoque experimental, se busca ofrecer una evaluación rigurosa y comparativa de los distintos esquemas de caché, proporcionando bases técnicas sólidas para la optimización de arquitecturas de memoria y almacenamiento.

1. El Problema

El proyecto aborda el diseño e implementación de un Simulador de Caché de Correspondencia Directa y Asociativa por Vías aplicado al contexto de Sistemas de Archivos. En el ámbito de la arquitectura de computadores y los sistemas operativos, la gestión eficiente de la jerarquía de memoria y el almacenamiento secundario es fundamental para mitigar la latencia de acceso a datos.

El problema central consiste en modelar computacionalmente el comportamiento de una memoria caché bajo dos esquemas de organización fundamentales:

- **Caché de Mapeo Directo (Direct-Mapped Cache):** Donde cada bloque

de memoria principal se mapea a una única línea específica en la caché.

- **Caché Asociativa por Vías (Set-Associative Cache):** Donde un bloque de memoria puede ubicarse en cualquiera de las vías disponibles dentro de un conjunto determinado, reduciendo los fallos por conflictos (*conflict misses*).

Para lograr esto, el simulador debe aceptar como entrada una secuencia de direcciones de memoria (provenientes de trazas reales o sintéticas) y procesarlas tomando en cuenta las características estructurales y operativas de dos sistemas de archivos diferentes a comparar. El objetivo final es evaluar, medir y comparar el rendimiento de los distintos esquemas de caché en función de métricas cuantitativas clave, tales como el número de aciertos (*hits*), fallos (*misses*) y la tasa de aciertos (*hit ratio*).

2. Técnicas Computacionales para su Solución

Para resolver el problema planteado, se ha desarrollado una solución en el lenguaje de programación C++ (específicamente utilizando el estándar C++17), aprovechando las ventajas de rendimiento del lenguaje y la flexibilidad de la Biblioteca Estándar de Plantillas (*Standard Template Library - STL*).

2.1. Entorno de Compilación y Configuración

El desarrollo y compilación del software se gestiona mediante un archivo de automatización (*Makefile*), configurado con optimizaciones del compilador GCC (`-O2`) y banderas estrictas de advertencia (`-Wall -Wextra`) para garantizar la robustez del código:

- **Compilador:** `g++`
- **Estándar:** C++17 (`-std=c++17`)
- **Módulos principales:** `main.cpp`, `cache.hpp`, `filesystem_profile.hpp` y `trace_generator.hpp`.

2.2. Estructuras de Datos en la STL

La implementación hace uso intensivo de contenedores avanzados de la STL para modelar la jerarquía y el estado de la memoria caché:

- `std::vector`: Utilizado para almacenar de forma dinámica los conjuntos de la caché, las líneas de caché por vía, los perfiles de los sistemas de archivos (`FileSystemProfile`) y los resultados de las ejecuciones de simulación

(`RunResult`). El uso de métodos como `reserve()` optimiza la asignación de memoria dinámica para evitar realocaciones innecesarias.

- `std::list`**:** Empleado para gestionar las listas de control de acceso y las políticas de reemplazo en las vías asociativas (como el seguimiento de bloques menos recientemente usados - LRU).
- `std::ifstream` **y flujos de E/S:** Utilizados para la lectura estructurada de archivos de trazas de direcciones de memoria y la persistencia de configuraciones.

2.3. Algoritmos de Simulación y Procesamiento

- **Descomposición de Direcciones:** El algoritmo toma cada dirección de memoria de 64 bits (o de tamaño estándar) de la traza de entrada y la descompone bit a bit mediante operaciones de desplazamiento (*bit shifting*) y máscaras (*bitwise AND*):
  - **Desplazamiento (*Offset*):** Bits inferiores para determinar la posición exacta del byte dentro del bloque.
  - **Índice de Conjunto (*Index*):** Bits intermedios para seleccionar el conjunto (*set*) correspondiente en la caché.
  - **Etiqueta (*Tag*):** Bits superiores almacenados en la línea de caché para verificar si la dirección buscada coincide con el bloque cargado.
- **Generación y Carga de Trazas:** El simulador soporta tanto la carga de trazas desde archivos externos (`loadTraceFromFile`) como la generación sintética de secuencias de direcciones con localidad de referencias ajustable (`generateTrace`), permitiendo simular patrones de acceso realistas de los sistemas de archivos bajo estudio.
- **Control de Aciertos y Fallos:** Iteración sobre cada acceso de la traza para comprobar la validez de la etiqueta y el estado de ocupación de las vías, actualizando contadores atómicos de aciertos (*Hits*) y fallos (*Misses*) y calculando la tasa de aciertos (*HitRatio*).

3. Análisis y Comparación de Resultados

El análisis de los resultados obtenidos a través del simulador permite evaluar el comportamiento de los esquemas de caché bajo las restricciones operativas de los dos sistemas de archivos comparados:

3.1. Evaluación de Métricas de Rendimiento

El simulador genera de manera tabular y estadística las siguientes métricas para cada combinación evaluada:

- **Número de Aciertos (*Hits*):** Cantidad de veces que la dirección solicitada se encontraba almacenada en la caché, evitando el acceso al sistema de archivos subyacente.
- **Número de Fallos (*Misses*):** Accesos que requirieron buscar la información en el nivel inferior de memoria.
- **Tasa de Aciertos (*Hit Ratio*):** Expresada como el cociente entre los aciertos totales y el número total de accesos de la traza, sirviendo como indicador principal de la eficiencia del esquema.

3.2. Comparación entre Esquemas (Mapeo Directo vs. Asociativo por Vías)

- **Impacto del Grado de Asociatividad:** Los resultados demuestran que incrementar el número de vías (*n-way set-associative*) reduce drásticamente los fallos por conflictos, especialmente en cargas de trabajo de sistemas de archivos donde múltiples archivos o metadatos (como inodos o tablas de asignación) comparten los mismos índices de caché.
- **Tamaño de Bloque y Localidad:** La variación en el tamaño de los bloques de los sistemas de archivos analizados muestra un compromiso directo (*trade-off*): bloques más grandes aprovechan mejor la localidad espacial para secuencias de lectura contiguas, pero incrementan la penalización por fallo y pueden generar mayor proporción de desalojos prematuros (*thrashing*) en cachés de menor capacidad total.
- **Conclusión Comparativa:** El informe generado por el simulador permite identificar cuantitativamente cuál esquema de caché maximiza el *hit ratio* para las características específicas de cada sistema de archivos evaluado, ofreciendo una base empírica para la optimización del rendimiento en arquitecturas de almacenamiento.